

PX4 Autonomous Flight SDK

Developer Specification & Architectural Standard

Version 1.0

Abstract

This document serves as the official Software Development Kit Specification (SDKS) for the companion computer-based autonomous flight architecture. If the original source realization (as archived in `px4_sdk_complete_2.zip`) were lost, this specification defines the interface boundaries, state machine structures, threading behaviors, and register mappings necessary to rebuild a functionally identical, binary-compatible SDK from scratch.

Prepared for Companion Computer Autonomous Missions & Guidance Systems

July 4, 2026

Contents

1	Overview	4
1.1	Purpose	4
1.2	SDK Goals	4
1.3	Design Philosophy	4
1.4	Why this SDK Exists	4
1.5	Supported Target Configurations	5
1.6	Dependencies	5
1.7	Compilation	5
1.8	Folder Structure	5
2	Architecture	6
2.1	Physical Architecture & Responsibilities	6
2.2	Thread Model	6
2.3	Telemetry Cache	6
2.4	Target Manager	6
2.5	Synchronization	7
2.6	Memory Ownership & Lifecycle	7
2.7	Architecture Diagram	7
3	Quick Start & Lifecycle	8
3.1	Two-Page Quick Start Guide	8
3.2	SDK Lifecycle Standard	10
3.3	Choosing the Right Function	10
4	Flight Philosophy	12
4.1	Closed-Loop Philosophy	12
4.2	The MANUAL_CONTROL Prohibition Rule	12
4.3	Safety and RC Override Priority	12
4.4	SDK Guarantees	12
4.5	What the SDK Does NOT Do	13
5	Coordinate Systems	14
5.1	North-East-Down (NED) Frame	14
5.2	Body Frame (Forward-Right-Down)	14
5.3	GPS Coordinates (WGS84)	14
5.4	Conversions and Transforms	14
6	Units	15

7 Thread Architecture	16
7.1 Heartbeat Thread	16
7.2 Telemetry Thread	16
7.3 Offboard Thread	16
7.4 Watchdog Thread	16
7.5 Mutex Lock Ordering & Deadlock Rules	16
8 Complete Function Directory (Quick Reference)	17
8.1 Purpose	17
8.2 Core	17
8.3 Flight	17
8.4 Navigation	17
8.5 Motion	18
8.6 Velocity	18
8.7 Speed	18
8.8 Altitude	18
8.9 Yaw	18
8.10 RC	19
8.11 Telemetry	19
8.12 Safety	19
8.13 Mission	19
8.14 Configuration	19
8.15 Logging	20
9 SDK Programming Model & Guidelines	21
9.1 Programming Model	21
9.2 Closed-Loop Philosophy	21
9.3 Implementation Guidelines	21
9.4 API Behaviour Specification	22
10 Mission Development Guide	39
10.1 Autonomy Command Sequence (ACS)	39
10.2 Mission Development Philosophy	39
10.3 State Machine Mission Design (Pickup Mission Example)	40
11 Common Mission Examples	43
12 Safety	44
13 Configuration	45
14 State Machine	46
15 Error Codes	47
15.1 SDK_OK	47
15.2 SDK_ERR_TIMEOUT	47
15.3 SDK_ERR_NOT_READY	47
15.4 SDK_ERR_INVALID_STATE	48
15.5 SDK_ERR_SERIAL	48
15.6 SDK_ERR_ACK_TIMEOUT	48
15.7 SDK_ERR_OFFBOARD_REJECTED	48
15.8 SDK_ERR_ARM_REJECTED	48
15.9 SDK_ERR_NO_GPS	49

15.10	SDK_ERR_EKF	49
15.11	SDK_ERR_RC_OVERRIDE	49
15.12	SDK_ERR_PARAMETER	49
15.13	SDK_ERR_INTERNAL	49
16	Thread Safety	50
17	PX4 Integration	51
18	Internal Operation	52
19	Best Practices	53
20	FAQ	54
21	Glossary	55
22	Appendix	56
22.1	System Return and Error State Codes	56
22.2	Which Function Should I Use?	56

Chapter 1

Overview

1.1 Purpose

This document defines the complete functional and binary specification for the PX4 Autonomous Flight SDK. The codebase implemented within `px4_sdk_complete_2.zip` is a single physical realization of this formal architecture. The prime goal of this SDK is to decouple high-level autonomous navigation loops (running on companion computer hardware) from the low-level hard real-time safety, attitude, and stabilization loops running on the flight controller.

1.2 SDK Goals

- **Decoupled Control:** Separate high-level mission logic from attitude control loops.
- **Zero-Allocation Safety:** Critical real-time communication paths must run without dynamic memory allocation.
- **Fault Isolation:** Companion computer failure must trigger deterministic failsafe states without destabilizing physical flight parameters.
- **Comprehensive Diagnostics:** Transparent tracing of every outbound command and incoming telemetry payload.

1.3 Design Philosophy

The SDK assumes a dual-processor computer topology. The Guidance Computer (NVIDIA Jetson / companion hardware) runs high-level mission state management, vision pipelines, and planning. The Flight Controller (Pixhawk / PX4) runs state estimation and hardware actuation. Communication is strictly defined via MAVLink v2.0 packets over high-speed serial links.

1.4 Why this SDK Exists

Standard MAVLink APIs can be complex and are prone to framing conflicts when accessed directly from asynchronous application loops. This SDK encapsulates this complexity within a thread-safe library with explicit execution timelines, allowing guidance developers and AI agents to program flight behavior with high safety margins.

1.5 Supported Target Configurations

- **Supported Hardware:** NVIDIA Jetson (Orin, Xavier, Nano), Raspberry Pi 4/5, Intel NUC.
- **Flight Controllers:** Pixhawk 4/5/6 Series, Cube Orange, Holybro Durandal.
- **Firmware:** PX4 Autopilot v1.13 and v1.14 (Stable).
- **MAVLink Version:** MAVLink v2.0.
- **Operating Systems:** Linux (Ubuntu 20.04/22.04 LTS with real-time kernel patches).

1.6 Dependencies

The SDK depends on:

- POSIX Threads (`pthread`) library for real-time scheduling.
- Standard C11 / C++17 compiler (`gcc-10` or higher).
- CMake 3.16+ build system.

1.7 Compilation

The SDK is built using CMake. To compile, the standard invocation is:

```
1 mkdir build && cd build
2 cmake -DCMAKE_BUILD_TYPE=Release ..
3 make -j$(nproc)
```

1.8 Folder Structure

The repository structure specified in `px4_sdk_complete_2.zip` is mapped as follows:

- `include/`: Core API headers containing the public definitions (`sdk_api_*.h`).
- `c_library_v2/`: Compiled MAVLink v2.0 message headers.
- `mav.c`: Main entry point implementing serial framing, telemetry decoding, and low-level loop hooks.
- `tests/`: Automated validation suits and synthetic flight telemetry caches.

Chapter 2

Architecture

2.1 Physical Architecture & Responsibilities

This standard specifies a master-follower model between the companion machine (Jetson/Guidance Computer) and the autopilot (Pixhawk/Follower).

- **Companion Computer (NVIDIA Jetson):** Executes guidance pipelines, SLAM algorithms, obstacle mapping, and target identification. It hosts the SDK instance.
- **Flight Controller (Pixhawk/PX4):** Runs low-level IMU-based attitude control, motor mixers, battery monitoring, and sensor fusion filters (EKF2).

2.2 Thread Model

To guarantee non-blocking, latency-bounded execution of public API functions, the SDK is structured around four high-priority POSIX threads:

1. **Heartbeat Thread:** Dispatches high-frequency system status markers (1Hz) to satisfy the PX4 connection verification requirements.
2. **Telemetry Thread:** Decodes raw byte streams from the UART buffer, verifies checksum integrity, and populates the Telemetry Cache.
3. **Offboard Stream Thread:** Actively packages position, velocity, and combined target reference sets into MAVLink frames at a steady 20Hz cadence.
4. **Watchdog Thread:** Tracks timestamp deviations across data boundaries, triggering state fallbacks on connection dropouts.

2.3 Telemetry Cache

A dynamic memory array acting as the absolute local reference for vehicle dynamics. Reads are non-blocking via read-write synchronization locks, ensuring zero-allocation performance.

2.4 Target Manager

Maintains the requested spatial state arrays (positions, velocities, headings). High-level command structures apply adjustments to the Target Manager registers, which are then streamed deterministically over offboard loops.

2.5 Synchronization

A strict, hierarchical mutex acquisition model is enforced statically across all sub-systems to eliminate thread race conditions and prevent deadlocks:

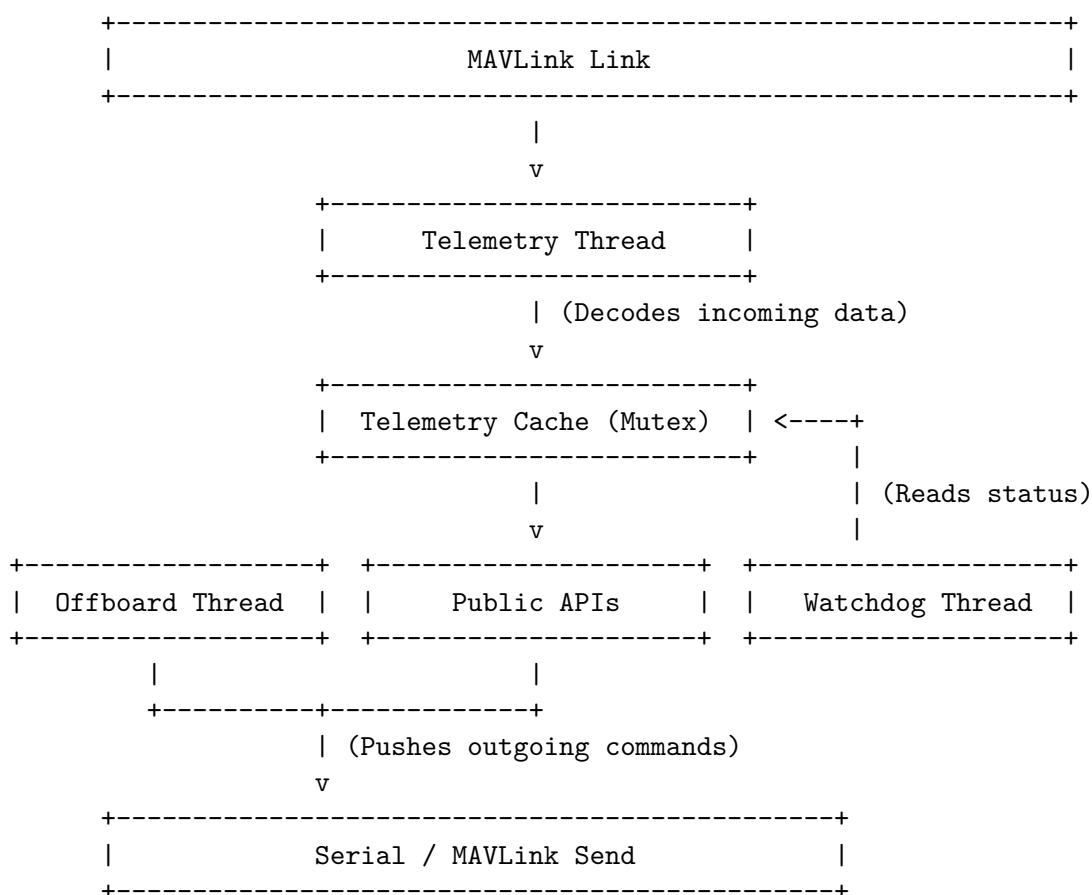
$$\text{Telemetry Mutex} \rightarrow \text{Target Mutex} \rightarrow \text{Serial Out Mutex} \quad (2.1)$$

2.6 Memory Ownership & Lifecycle

To guarantee high flight envelope safety, memory fragmentation is avoided. All structural blocks, thread arguments, and message structures are statically pre-allocated at initialization. Memory management tracks three phases: instantiation, thread-join shutdown, and socket release.

2.7 Architecture Diagram

The diagram below details the data flow and locking boundaries between the threads and the system memory:



Chapter 3

Quick Start & Lifecycle

3.1 Two-Page Quick Start Guide

For developers integrating custom automated flight plans, the SDK abstractly manages high-frequency message packaging and heartbeat constraints. The minimum application logic requires setting parameters, initiating takeoff, commanding spatial changes, and safe landing.

Below is a complete, runnable quick start script designed to deploy a basic automated tracking path:

```
1 #include "include/sdk_api_core.h"
2 #include "include/sdk_api_flight.h"
3 #include "include/sdk_api_navigation.h"
4 #include <stdio.h>
5 #include <unistd.h>
6
7 int main(void)
8 {
9     // Initialize serial communication and spawn system threads
10    start("/dev/ttyACM0", 921600);
11
12    // Arm the aircraft propulsion systems
13    arm();
14
15    // Climb vertically to a pre-defined safety altitude (1.50 meters)
16    takeoff(150);
17
18    // Navigate to target GPS latitude and longitude values
19    gps_go(22.5726, 88.3639);
20
21    // Establish dynamic target lock and hover in place for 3000 ms
22    hover(3000);
23
24    // Perform structured vertical landing descent sequence
25    land();
26
27    // Halt message loops, join background worker threads, and close port
28    stop();
29
30    return 0;
31 }
```

Sequential Operations Breakdown

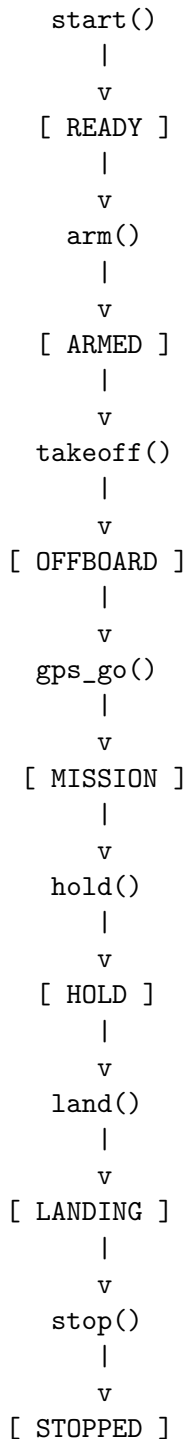
1. **Initialize:** The call to `start()` opens the specified physical serial device node (e.g. `/dev/ttyACM0`), sets terminal line attributes to 921600 baud, initializes structural locking layers, and brings

up worker threads.

2. **Arm:** `arm()` evaluates current pre-flight check signals. If GPS, battery, and EKF2 registers indicate healthy margins, a localized arming packet is written to the physical line.
3. **Takeoff:** `takeoff()` transitions the vehicle system from static positioning into vertical ascent. The function blocks until the vehicle enters offboard control and altitude measurements verify climb completion.
4. **Fly:** `gps_go()` shifts local target vectors towards global parameters, triggering trajectory navigation routines.
5. **Hover:** `hover()` clears active travel vectors and locks local position markers in place.
6. **Land:** `land()` dispatches auto-land commands. The routine monitors tracking estimators, returning control only when motor shut-down bits are detected.
7. **Shutdown:** `stop()` terminates loop streaming and cleanly closes serial file handles.

3.2 SDK Lifecycle Standard

The state transition flow below maps the logical prerequisites and outputs associated with main system commands. Transitions must follow verified, predictable pathways:



3.3 Choosing the Right Function

To streamline development workflows and prevent search delays, this lookup guide acts as a direct reference matching operational goals to dedicated APIs:

Developer Goal	Desired Dynamic Control	Target API Call
Takeoff automatically	Altitude hold climbing loop	<code>takeoff()</code>
Travel to coordinate set	Global map waypoint translation	<code>gps_go()</code>
Travel relative to home coordinate	Cartesian translation vector offset	<code>local_go()</code>
Advance relative to heading	Forward-lateral-down offset	<code>body_go()</code>
Direct velocity vector mapping	Closed-loop dynamic vectoring	<code>set_velocity()</code>
Alter active flight path elevation	Altitude target override loop	<code>set_alt()</code>
Orient true angle relative to North	Heading angle rotation lock	<code>yaw_to()</code>
Orient rotational rate changes	Continuous angular rotational speed	<code>yaw_rate()</code>
Suspend travel instantly	Vector braking	<code>emergency_brake()</code>
Trigger return-to-land routine	Automatic landing coordinates return	<code>rtl()</code>

Table 3.1: Dynamic Function Mapping Matrix

Chapter 4

Flight Philosophy

4.1 Closed-Loop Philosophy

Autonomy on the Jetson relies strictly on state-dependent setpoints. High-frequency stabilization loops (attitude, rate, and sensor feedback) must remain on the flight controller. The companion computer provides position, velocity, and trajectory targets.

4.2 The MANUAL_CONTROL Prohibition Rule

No autonomous guidance application is permitted to construct or transmit raw `MANUAL_CONTROL` (RC joysticks emulator) messages to the autopilot. This prevents unpredictable pilot stick scaling issues and bypasses flight controller safe envelope restrictions. Flight targets must instead be requested using formal coordinate frame sets.

4.3 Safety and RC Override Priority

The manual radio control receiver (RC) connected to the flight controller always maintains absolute safety priority. At any instant during autonomous companion computer guidance (Offboard Mode), a pilot stick movement past a configured threshold (typically 10% deadband deflection) immediately forces the flight controller to drop Offboard control and transition back to manual Position or Altitude Hold. The SDK must handle this transition gracefully by entering a read-only tracking state.

4.4 SDK Guarantees

To ensure predictable performance in critical environments, the SDK establishes seven mathematical and behavioral guarantees across all public APIs:

- **Closed-Loop Operation:** Dynamic spatial tracking targets are continuously managed and verified against real-time sensor feedback.
- **Autopilot-Driven Stabilization:** High-frequency attitude and rate control loops remain separated on the real-time autopilot processor.
- **Telemetry-Confirmed Completion:** Blocking navigation calls query dynamic state caches to confirm target arrival prior to returning control.

- **No RC Emulation:** Avoids using raw joystick emulation commands, ensuring flight envelope protection is managed natively by PX4.
- **Thread Safety:** Internal registers, cache locations, and serial output buffers are structurally isolated using strict lock hierarchies.
- **Dynamic Offboard Compatibility:** Outbound setpoint streaming strictly conforms to the minimum 2Hz offboard timeout threshold (operating at 20Hz).
- **State Machine Validation:** Commands are checked against the active SDK state machine to block illegal transitions (e.g., preventing arming commands during active flight).

4.5 What the SDK Does NOT Do

To prevent system failures caused by architectural scope creep, this SDK is designed as a raw flight control wrapper. The library explicitly does NOT perform any of the following tasks:

- **Computer Vision:** Does not capture, process, or filter raw camera matrices.
- **Object Detection:** Does not execute target tracking, identification, or neural network inference loops.
- **Mission Orchestration:** Does not compute task allocation, waypoint generation, or route optimization.
- **Path Planning:** Does not calculate local trajectory curves or execute SLAM maps.
- **Obstacle Avoidance:** Does not process LIDAR, depth, or ultrasound data to dodge obstacles.
- **Search Execution:** Does not contain automated search patterns or coordinate mapping routines.
- **Actuator / Servo Handling:** Does not handle generic mechanical payload release triggers or servo outputs.
- **AI Decision-Making:** Does not evaluate mission priorities, coordinate assets, or handle target acquisition.

Chapter 5

Coordinate Systems

5.1 North-East-Down (NED) Frame

The local guidance coordinate system used by the autopilot is local NED (North, East, Down). In this frame:

- The X-axis points North.
- The Y-axis points East.
- The Z-axis points Down (making altitude negative relative to the reference point).

5.2 Body Frame (Forward-Right-Down)

The vehicle relative coordinate system:

- The X-axis points Forward through the nose of the aircraft.
- The Y-axis points Right through the wing/propeller assembly.
- The Z-axis points Down through the floor.

5.3 GPS Coordinates (WGS84)

Global positions are defined using the WGS84 ellipsoid:

- **Latitude:** Degrees North/South.
- **Longitude:** Degrees East/West.
- **Altitude:** Meters above mean sea level (MSL) or above the takeoff point (Relative Altitude).

5.4 Conversions and Transforms

The orientation of the body frame is converted to quaternions $\mathbf{q} = [w, x, y, z]^T$ or Euler angles (Yaw ψ , Pitch θ , Roll ϕ). Yaw angles use a range of $[-\pi, \pi]$, where 0 is North and positive rotation is clockwise (East).

The rotation matrix R_{body}^{ned} from Body to NED coordinates is defined as:

$$R_{body}^{ned} = \begin{bmatrix} \cos \theta \cos \psi & \sin \phi \sin \theta \cos \psi - \cos \phi \sin \psi & \cos \phi \sin \theta \cos \psi + \sin \phi \sin \psi \\ \cos \theta \sin \psi & \sin \phi \sin \theta \sin \psi + \cos \phi \cos \psi & \cos \phi \sin \theta \sin \psi - \sin \phi \cos \psi \\ -\sin \theta & \sin \phi \cos \theta & \cos \phi \cos \theta \end{bmatrix} \quad (5.1)$$

Chapter 6

Units

To prevent mathematical interpretation errors, all parameters passed to or from the SDK must use the exact SI units defined below:

- **Distance / Displacement:** Meters [m], accurate to standard float resolution.
- **Velocity:** Meters per second [m/s].
- **Angle / Attitude:** Radians [rad] (except where specifically configured for user-facing coordinate setups).
- **Time:** Seconds [s] for physical tracking metrics; Milliseconds [ms] for thread wait definitions.
- **GPS Coordinates:** Latitude/Longitude in decimal degrees (WGS84 ellipsoid).

Chapter 7

Thread Architecture

7.1 Heartbeat Thread

Generates standard HEARTBEAT signals to notify PX4 of the active system presence. This operates synchronously on a fixed 1Hz timer cycle.

7.2 Telemetry Thread

Performs blocking reads on the serial physical link, unpacks MAVLink frame envelopes, and updates the Telemetry Cache with raw state indicators.

7.3 Offboard Thread

Executes coordinate trajectory packet generation at a continuous 20Hz cycle. If updates are delayed past a 500ms timeout boundary, offboard command status fails back to stable hold loops.

7.4 Watchdog Thread

Tracks time metrics across active communication lines. If incoming packets from PX4 cease for more than 500ms, a link degradation fault is triggered.

7.5 Mutex Lock Ordering & Deadlock Rules

To prevent circular wait states and lock inversion, the SDK enforces a strict, hierarchical lock sequence:

Telemetry Mutex → Target Mutex → Serial Out Mutex (7.1)

No thread is permitted to acquire a higher-order lock while holding a lower-order lock. Any violations will trigger the SDK assertion system.

Chapter 8

Complete Function Directory (Quick Reference)

8.1 Purpose

This chapter is NOT documentation. It is simply a master directory. Every function appears exactly once in the format: `function_name(arguments)`, categorized by system subsystem blocks.

8.2 Core

```
start()
stop()
restart()
sdk_ready()
sdk_version()
```

8.3 Flight

```
arm()
disarm()
takeoff(float altitude_m)
land()
land(float descent_rate)
rtl()
hover()
hover(int duration_ms)
emergency_brake()
set_mode(flight_mode_t mode)
```

8.4 Navigation

```
gps_go(double latitude,
        double longitude,
        float altitude_m = CURRENT_ALTITUDE,
        float speed_mps = CURRENT_SPEED)
```

```
local_go(float north,  
         float east,  
         float down)
```

```
body_go(float forward,  
        float right,  
        float down)
```

8.5 Motion

```
forward(float meters)  
backward(float meters)  
left(float meters)  
right(float meters)  
up(float meters)  
down(float meters)
```

8.6 Velocity

```
set_velocity(float vx,  
            float vy,  
            float vz)
```

```
stop_velocity()
```

8.7 Speed

```
set_speed(float speed)  
get_speed()
```

8.8 Altitude

```
set_alt(float altitude)  
change_alt(float delta)  
hold_alt()  
get_alt()
```

8.9 Yaw

```
yaw(float angle)  
yaw_to(float heading)  
yaw_rate(float rate)  
get_yaw()
```

8.10 RC

```
rc_override_enabled()  
wait_for_rc_takeover()  
clear_rc_takeover()
```

8.11 Telemetry

```
get_location()  
get_local_position()  
get_velocity()  
get_attitude()  
get_roll()  
get_pitch()  
get_yaw()  
get_heading()  
get_battery()  
get_satellites()  
get_mode()  
get_arm_state()
```

8.12 Safety

```
is_connected()  
gps_ok()  
battery_ok()  
ekf_ok()  
health_ok()  
telemetry_ok()  
failsafe_active()
```

8.13 Mission

```
wait(int milliseconds)  
pause_mission()  
resume_mission()  
abort_mission()
```

8.14 Configuration

```
set_parameter()  
get_parameter()  
load_config()  
save_config()  
reset_config()
```

8.15 Logging

```
enable_logging()  
disable_logging()  
log_info()  
log_warning()  
log_error()
```

This chapter serves as the master API directory. Developers should first locate the required function here, then refer to the detailed API Reference chapter for complete documentation, behavior, parameters, return values, blocking characteristics, state transitions, and examples.

Chapter 9

SDK Programming Model & Guidelines

9.1 Programming Model

Autonomy loops on the Jetson computer interact asynchronously with the flight controller's state variables. To construct robust software programs, developers must follow the guidelines detailed in this chapter. The SDK uses POSIX-level synchronization boundaries to update memory without timing drift.

9.2 Closed-Loop Philosophy

Guidance processes are strictly split into high-latency trajectory tracking (on the companion computer) and hard-realtime motor-stabilization tracking (on the PX4 flight controller). To preserve stability, direct actuation modifications are forbidden, and commands must instead be formulated using local position offsets or velocity vectors.

9.3 Implementation Guidelines

When writing autopilot navigation scripts, developers must:

1. Verify that the EKF2 filters have converged (the global position estimation covariance remains low).
2. Keep the heartbeat timer running on a non-blocking thread to satisfy PX4's 1Hz heartbeat deadline.
3. Poll telemetry registers using standard, mutually exclusive reads to avoid thread race conditions.

9.4 API Behaviour Specification

sdk_core_init

Prototype

```
int sdk_core_init(const char* device, int baudrate);
```

Header

```
#include <sdk/sdk_core.h>
```

Module

Core

Purpose

Initializes the serial configuration buffers, configures file descriptors, and registers the underlying thread pools.

Parameters

- `device` (Type: `const char*`, Units: N/A, Valid Range: Valid system path, e.g., `"/dev/ttyTHS1"`). Default Behavior: None. Required parameter.
- `baudrate` (Type: `int`, Units: Baud, Valid Range: 9600 to 1000000). Default Behavior: None. Required parameter.

Returns

0 on success, negative value mapped to system error on initialization failure.

Execution Behaviour

Blocking.

Internal Algorithm

1. Verifies that the internal state register is set to `STATE_UNINITIALIZED`.
2. Opens the target serial device node.
3. Sets the requested serial terminal parameters (baud rate, 8N1 framing, no flow control).
4. Initializes POSIX mutex clocks and spawns the worker threads.

Telemetry Used

None.

PX4 Commands Sent

None.

Thread Interaction

Spawns and configures all background companion threads.

State Machine Effects

Transitions the SDK state from `STATE_UNINITIALIZED` to `STATE_INITIALIZED`.

Failure Conditions

- Invalid or non-existent serial device path.
- Insufficient system permissions.

Warnings

Calling this function multiple times without an intermediate call to `sdk_core_shutdown` will result in undefined locking behaviors.

Performance Notes

Usually completes within 50ms.

Example

```
1 if (sdk_core_init("/dev/ttyTHS1", 921600) != 0) {  
2     printf("Hardware link failed.\n");  
3 }
```

See Also

`sdk_core_shutdown`

sdk_core_shutdown

Prototype

```
void sdk_core_shutdown(void);
```

Header

```
#include <sdk/sdk_core.h>
```

Module

Core

Purpose

Gracefully terminates background worker threads and closes open serial file descriptors.

Parameters

None.

Returns

None.

Execution Behaviour

Blocking.

Internal Algorithm

1. Raises termination flags across all background POSIX worker threads.
2. Invokes `pthread_join` on each thread to verify clean closure.
3. Closes the open serial file descriptor.

Telemetry Used

None.

PX4 Commands Sent

None.

Thread Interaction

Terminates and joins the background thread pool.

State Machine Effects

Transitions state back to `STATE_UNINITIALIZED`.

Failure Conditions

Hanging loops can occur if a worker thread is blocked on a systemic read call.

Warnings

Must only be called after offboard maneuvers are safely terminated and the aircraft is landed.

Performance Notes

Thread joining can take up to 1 second depending on active hardware states.

Example

```
1 sdk_core_shutdown();
```

See Also

`sdk_core_init`

sdk_flight_takeoff

Prototype

```
int sdk_flight_takeoff(float target_altitude);
```

Header

```
#include <sdk/sdk_flight.h>
```

Module

Flight

Purpose

Commands the aircraft to arm its propulsion systems and perform an autonomous climbing takeoff.

Parameters

- `target_altitude` (Type: `float`, Units: Meters [m], Valid Range: 1.0f to 50.0f). Default Behavior: None. Required parameter.

Returns

0 on successful hover establishment, negative error code on rejection or timing failures.

Execution Behaviour

Blocking.

Internal Algorithm

1. Begins streaming zero-velocity offboard setpoints to satisfy the PX4 interface constraint.
2. Requests offboard mode activation and verifies confirmation via incoming telemetry.
3. Dispatches arming command (`MAV_CMD_COMPONENT_ARM_DISARM`) to flight controller.
4. Dispatches vertical position target climbing commands and monitors relative altitude until target is achieved within a $\pm 15\text{cm}$ deadband.

Telemetry Used

- Read current state to monitor armed flags and coordinate tracking values.
- Reads relative altitude to verify takeoff ascent tracking loop convergence.

PX4 Commands Sent

- `MAV_CMD_DO_SET_MODE`
- `MAV_CMD_COMPONENT_ARM_DISARM`
- `SET_POSITION_TARGET_LOCAL_NED`

Thread Interaction

Interacts with the Offboard Stream Thread to update the target coordinate state arrays.

State Machine Effects

Transitions the SDK state from `STATE_INITIALIZED` to `STATE_TAKING_OFF`, then to `STATE_OFFBOARD_STABLE`.

Failure Conditions

- Loss of satellite positioning reference.
- Mode activation command rejected by PX4 safety checkers.

Warnings

Takeoff actions are hazardous if GPS dilution metrics indicate high covariance values.

Performance Notes

Climb times scale linearly with target altitude (typically ascending at 1.0 m/s to 1.5 m/s).

Example

```
1 if (sdk_flight_takeoff(5.0f) != 0) {  
2     printf("Takeoff maneuver aborted!\n");  
3 }
```

See Also

sdk_flight_land, sdk_flight_hover

sdk_flight_land

Prototype

```
int sdk_flight_land(void);
```

Header

```
#include <sdk/sdk_flight.h>
```

Module

Flight

Purpose

Instructs the vehicle to descend vertically at its current horizontal coordinate and land.

Parameters

None.

Returns

0 on successful touchdown and disarming, negative error code on mode rejection.

Execution Behaviour

Blocking.

Internal Algorithm

1. Queries telemetry values to record current x and y positions.
2. Sends a mode change request command to PX4 to activate the landing loop (MAV_CMD_DO_SET_MODE).
3. Polls the vehicle armed status, returning only when touchdown checks are complete and the motors disarm.

Telemetry Used

- Reads vehicle status to monitor arm status and landing detection bits.

PX4 Commands Sent

- MAV_CMD_DO_SET_MODE with landing configuration values.

Thread Interaction

Suspends targeted tracking updates to let internal autopilot routines control vertical descent.

State Machine Effects

Transitions the SDK state to STATE_LANDING, and then to STATE_INITIALIZED on motor shutdown.

Failure Conditions

- Mode command rejected by flight controller.
- Autopilot fails to complete automated landing checks.

Warnings

Ground surfaces must be flat and free of obstacles before landing commands are executed.

Performance Notes

Descent rates are controlled internally by PX4 (typically descending at a steady 1.0 m/s).

Example

```
1 if (sdk_flight_land() != 0) {  
2     printf("Autonomous landing failed!\n");  
3 }
```

See Also

sdk_flight_takeoff

sdk_flight_hover

Prototype

```
int sdk_flight_hover(void);
```

Header

```
#include <sdk/sdk_flight.h>
```

Module

Flight

Purpose

Commands the aircraft to pause its active route and hover at its current coordinates.

Parameters

None.

Returns

0 on successful state transition, negative error code if offboard targets are uninitialized.

Execution Behaviour

Non-blocking.

Internal Algorithm

1. Locks the Telemetry Cache and reads current local position values (x, y, z) .
2. Overwrites the active Target Manager coordinate arrays with these current coordinates.
3. Commands the Offboard Thread to stream these coordinates at 20Hz.

Telemetry Used

- Reads local position coordinates (x, y, z) .

PX4 Commands Sent

- Continuous streaming of coordinates via SET_POSITION_TARGET_LOCAL_NED.

Thread Interaction

Interacts with the Offboard Stream Thread to lock coordinate targeting loops.

State Machine Effects

Sets state to STATE_OFFBOARD_STABLE.

Failure Conditions

- Telemetry thread is offline or values in the cache are stale.

Warnings

None.

Performance Notes

Updates targets within 5ms of invocation.

Example

```
1 sdk_flight_hover();
```

See Also

sdk_flight_brake

sdk_flight_brake

Prototype

```
int sdk_flight_brake(void);
```

Header

```
#include <sdk/sdk_flight.h>
```

Module

Flight

Purpose

Aborts active navigation routes and applies lateral braking commands to decelerate the vehicle to a stop.

Parameters

None.

Returns

0 on successful brake initiation, negative value if the flight modes are not in offboard control.

Execution Behaviour

Non-blocking.

Internal Algorithm

1. Overwrites current setpoints inside the Target Manager to target zero velocity.
2. Configures bitmask filters to bypass coordinate calculations.
3. Flags the active stream thread to prioritize velocity over position values.

Telemetry Used

None.

PX4 Commands Sent

- Zero velocity commands sent via SET_POSITION_TARGET_LOCAL_NED.

Thread Interaction

Interrupts the Target Manager thread to clear pending position coordinates.

State Machine Effects

None.

Failure Conditions

- The flight controller is in a manual fallback state.

Warnings

Abrupt braking commands trigger sudden, heavy pitch and roll changes to shed speed.

Performance Notes

Braking commands are dispatched to the serial output buffer in less than 1ms.

Example

```
1 sdk_flight_brake();
```

See Also

sdk_flight_hover

sdk_nav_goto_gps

Prototype

```
int sdk_nav_goto_gps(double lat, double lon, float alt_msl);
```

Header

```
#include <sdk/sdk_navigation.h>
```

Module

Navigation

Purpose

Commands the aircraft to travel to a designated GPS coordinate.

Parameters

- `lat` (Type: double, Units: Degrees, Valid Range: -90.0 to 90.0). Default Behavior: None. Required parameter.
- `lon` (Type: double, Units: Degrees, Valid Range: -180.0 to 180.0 (or 180.0)). Default Behavior: None. Required parameter.
- `alt_msl` (Type: float, Units: Meters [m], Valid Range: 0.0f to 5000.0f). Default Behavior: None. Required parameter.

Returns

0 if waypoint is reached successfully, negative error code if command is rejected or times out.

Execution Behaviour

Blocking.

Internal Algorithm

1. Dispatches the coordinate values to PX4 via standard waypoint navigation protocols.
2. Translates targets into the local coordinate system using the active EKF2 map origin.
3. Monitors telemetry values to check distance-to-target registers, returning once the vehicle is within the geofence tolerance.

Telemetry Used

- Reads vehicle global coordinate estimates to calculate target distance.

PX4 Commands Sent

- MAV_CMD_DO_REPOSITION

Thread Interaction

Interacts with the Telemetry Thread to poll active state vectors.

State Machine Effects

None.

Failure Conditions

- GPS coordinates are degraded or invalid.
- The aircraft encounters a safety geofence boundary.

Warnings

Always check local airspace and geofence restrictions before calling global GPS maneuvers.

Performance Notes

Loop polling is executed at a steady 10Hz frequency to conserve companion computer resources.

Example

```
1 if (sdk_nav_goto_gps(47.397742, 8.545593, 450.0f) != 0) {  
2     printf("Navigation failed!\n");  
3 }
```

See Also

sdk_nav_goto_local

sdk_nav_goto_local

Prototype

```
int sdk_nav_goto_local(float x, float y, float z);
```

Header

```
#include <sdk/sdk_navigation.h>
```

Module

Navigation

Purpose

Commands travel to a designated local Cartesian point relative to home.

Parameters

- *x* (Type: `float`, Units: Meters [m], Valid Range: `-5000.0f` to `5000.0f`). Default Behavior: None. Required parameter.
- *y* (Type: `float`, Units: Meters [m], Valid Range: `-5000.0f` to `5000.0f`). Default Behavior: None. Required parameter.
- *z* (Type: `float`, Units: Meters [m], Valid Range: `-500.0f` to `0.0f`). Default Behavior: None. Required parameter.

Returns

0 on successful arrival, negative error code if offboard streaming drops out.

Execution Behaviour

Blocking.

Internal Algorithm

1. Sets active tracking parameters inside the Target Manager thread.
2. Updates offboard position command values.
3. Monitors telemetry estimates, blocking until local positions match targets within $\pm 20\text{cm}$.

Telemetry Used

- Reads current local positions (x, y, z) .

PX4 Commands Sent

- `SET_POSITION_TARGET_LOCAL_NED`

Thread Interaction

Integrates with the Offboard Stream Thread to continuously publish updated targets.

State Machine Effects

None.

Failure Conditions

- Command timeout matches limit threshold values.
- Failsafe mode transitions are triggered during travel.

Warnings

The local coordinate reference frame is established on boot and does not adapt dynamically if the EKF origin drifts.

Performance Notes

Maneuver durations vary depending on speed limits and aerodynamic conditions.

Example

```
1 sdk_nav_goto_local(10.0f, -5.0f, -4.0f);
```

See Also

`sdk_nav_goto_gps`

sdk_vel_set

Prototype

```
int sdk_vel_set(float vx, float vy, float vz);
```

Header

```
#include <sdk/sdk_velocity.h>
```

Module

Velocity

Purpose

Streams continuous 3D velocity vectors (V_x, V_y, V_z) to navigate the aircraft.

Parameters

- v_x (Type: float, Units: Meters per second [m/s], Valid Range: -15.0f to 15.0f). Default Behavior: None. Required parameter.
- v_y (Type: float, Units: Meters per second [m/s], Valid Range: -15.0f to 15.0f). Default Behavior: None. Required parameter.
- v_z (Type: float, Units: Meters per second [m/s], Valid Range: -5.0f to 5.0f). Default Behavior: None. Required parameter.

Returns

0 on successful target transmission, negative error code if offboard control is lost.

Execution Behaviour

Non-blocking.

Internal Algorithm

1. Sets target velocity vectors inside the Target Manager coordinate registry.
2. Instructs the Offboard Stream Thread to format and packetize the command payloads.
3. Dispatches updated telemetry packets containing the target velocity coordinates.

Telemetry Used

None.

PX4 Commands Sent

- SET_POSITION_TARGET_LOCAL_NED (with position commands bypassed).

Thread Interaction

Instructs the Offboard Stream Thread to actively update the velocity command output registers.

State Machine Effects

None.

Failure Conditions

- Command request sent when the vehicle is not in offboard control.

Warnings

Autopilot systems do not run obstacle collision avoidance routines when executing manual off-board velocity commands.

Performance Notes

Applies command changes to the physical interface in under 20ms.

Example

```
1 sdk_vel_set(1.5f, 0.0f, -0.2f);
```

See Also

sdk_flight_brake

sdk_yaw_set_target

Prototype

```
int sdk_yaw_set_target(float target_yaw);
```

Header

```
#include <sdk/sdk_yaw.h>
```

Module

Yaw

Purpose

Sets an absolute orientation heading target for the vehicle.

Parameters

- `target_yaw` (Type: `float`, Units: Radians [rad], Valid Range: $-\pi$ to π). Default Behavior: None. Required parameter.

Returns

0 on successful setpoint registration, negative error code if telemetry checks fail.

Execution Behaviour

Non-blocking.

Internal Algorithm

1. Modifies the active heading setpoint inside the target tracker arrays.
2. Bypasses angular velocity trackers, locking the Target Manager thread to apply absolute angle corrections.
3. Embeds the updated heading value in all downstream offboard setpoint packets.

Telemetry Used

None.

PX4 Commands Sent

- Yaw targets mapped into outbound `SET_POSITION_TARGET_LOCAL_NED` frames.

Thread Interaction

Interacts with the Offboard Stream Thread to coordinate heading setpoint commands.

State Machine Effects

None.

Failure Conditions

- Target value exceeds valid coordinate boundaries.

Warnings

Sudden, large heading changes can degrade optical flow tracking metrics and cause lateral drift.

Performance Notes

Changes are processed and applied in under 1ms.

Example

```
1 sdk_yaw_set_target(1.5707f); // Point true East
```

See Also

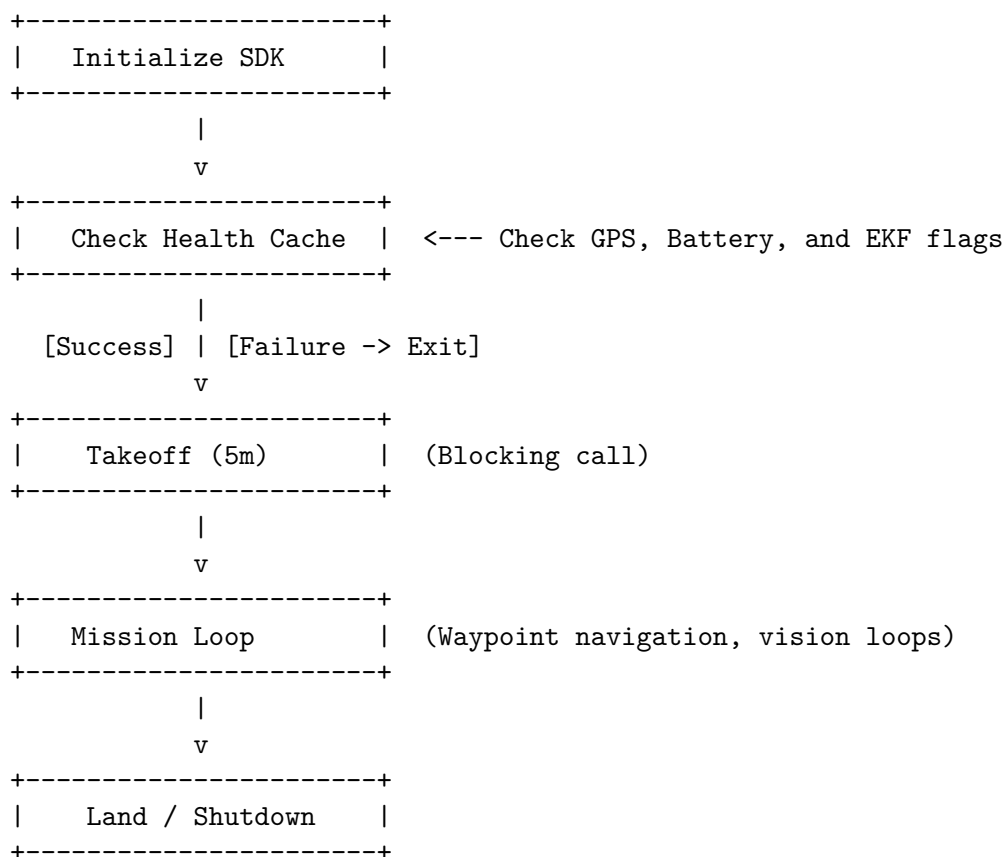
sdk_nav_goto_local

Chapter 10

Mission Development Guide

10.1 Autonomy Command Sequence (ACS)

To construct flight programs with high reliability, developers must adhere to the **Autonomy Command Sequence (ACS)**. High-level mission threads must coordinate execution checkpoints asynchronously to safely guide the vehicle during critical flight phases.



10.2 Mission Development Philosophy

An essential rule of autonomous drone operations: **The SDK only provides primitive, highly reliable flight actions. High-level mission logic is responsible for combining these primitives.**

None of the search, detection, tracking, or aligning logic belongs inside the SDK core library. By keeping the SDK core lightweight and clean of specific operational patterns, flight safety standards remain decoupled from application layers:

[Search] -> [Detect] -> [Track] -> [Align] -> [Descend] -> [Pickup]

Good Practices

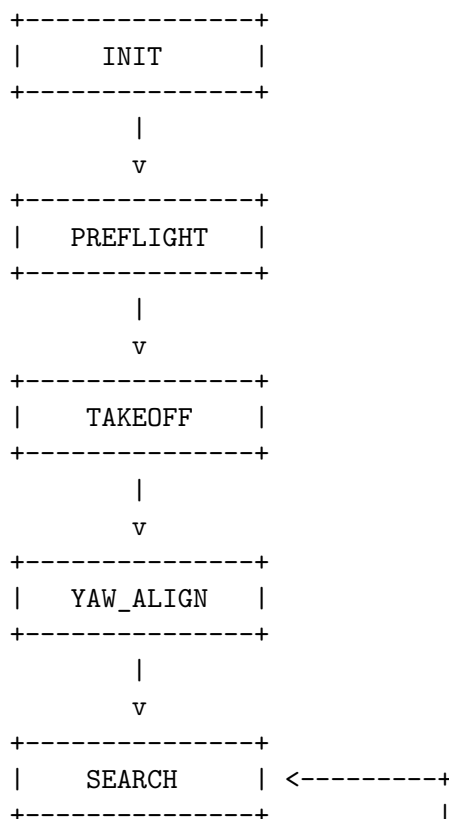
- **Always Verify Altitude Values:** Never command absolute elevation updates without first confirming a healthy GPS lock and checking home altitude.
- **Constant Command Intervals:** When streaming offboard setpoints, maintain a consistent 20Hz update loop. A pause longer than 500ms will trigger a PX4 offboard timeout and cause the vehicle to enter failsafe mode.

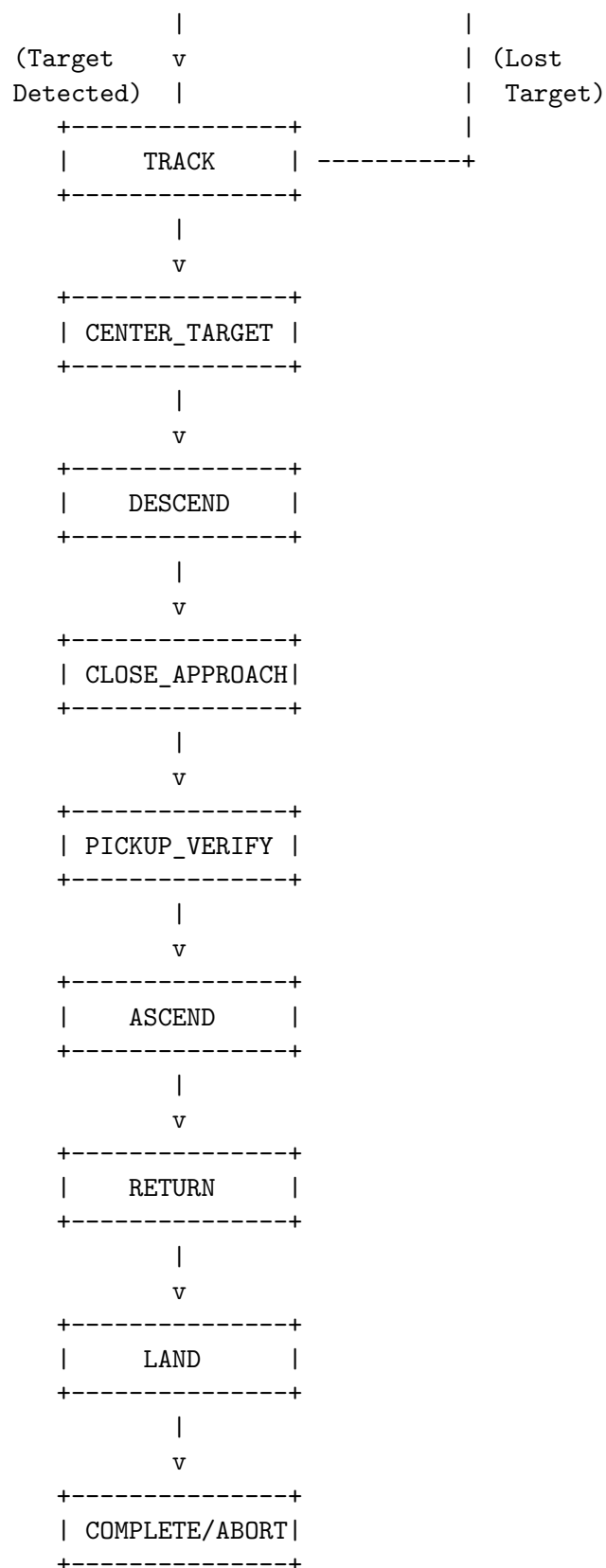
Bad Practices

- **Tight Spin-Loops:** Do not query telemetry values inside hot while-loops without adding thread sleep commands; doing so will saturate the companion CPU cores.
- **Direct System Override:** Attempting to force setpoints while the RC switch indicates a manual mode selection will fail.

10.3 State Machine Mission Design (Pickup Mission Example)

High-level mission scripts must be structured explicitly as standard Finite State Machines (FSMs) rather than sequential, nested conditional loops. Each state should only invoke SDK primitives, keeping decision logs cleanly isolated from flight behaviors:





Pickup State Matrix and Primitive Mapping

- **INIT:** Initializes the hardware serial port using `start("/dev/ttyTHS1", 921600)`.
- **PREFLIGHT:** Polls cache data (`gps_ok()`, `ekf_ok()`) to confirm baseline safety readiness.

- **TAKEOFF:** Executes a vertical climb target via `takeoff(250)`.
- **YAW_ALIGN:** Sets initial directional search heading using `yaw_to(180.0f)`.
- **SEARCH:** Scours the zone relative to local coordinate origin limits using `gps_go()`.
- **TRACK:** Dynamically adjusts velocity tracking vectors via companion computer target vision coordinates using `set_velocity(vx, vy, vz)`.
- **CENTER_TARGET:** Matches local coordinate points relative to the payload using `local_go(x, y, z)`.
- **DESCEND:** Commands altitude reductions down to payload pickup height using `set_alt(0.5f)`.
- **CLOSE_APPROACH:** Coordinates precision landing descent metrics using `body_go(0, 0, 0.2f)`.
- **PICKUP_VERIFY:** Blocks and waits while checking gripper payload confirmation signals.
- **ASCEND:** Executes vertical climb targets with payload attached using `set_alt(3.0f)`.
- **RETURN:** Traverses back to local coordinate base using `local_go(0, 0, -3.0f)`.
- **LAND:** Lands vehicles cleanly on final arrival coordinates via `land()`.
- **COMPLETE / ABORT:** Disarms propellers using `disarm()` and shuts down serial loops cleanly using `stop()`.

Chapter 11

Common Mission Examples

The following example code implements a classic autonomous waypoint navigation loop:

```
1 #include "include/sdk_api_core.h"
2 #include "include/sdk_api_flight.h"
3 #include "include/sdk_api_navigation.h"
4 #include <stdio.h>
5 #include <unistd.h>
6
7 void execute_waypoint_mission() {
8     printf("Starting Waypoint Mission...\n");
9
10    // Waypoint coordinates: North (m), East (m), Down (m)
11    float waypoints[3][3] = {
12        {10.0f, 0.0f, -5.0f},    // Point 1: 10m North, 5m altitude
13        {10.0f, 10.0f, -5.0f},  // Point 2: 10m North, 10m East
14        {0.0f, 0.0f, -3.0f}     // Point 3: Return to home, 3m altitude
15    };
16
17    for (int i = 0; i < 3; i++) {
18        printf("Navigating to Waypoint %d: N: %.1f, E: %.1f, D: %.1f\n",
19            i + 1, waypoints[i][0], waypoints[i][1], waypoints[i][2]);
20
21        int res = sdk_nav_goto_local(waypoints[i][0], waypoints[i][1], waypoints[i][2]);
22        if (res != 0) {
23            printf("Waypoint navigation aborted at point %d. Code: %d\n", i + 1, res);
24            return;
25        }
26        sleep(2); // Dwell at waypoint
27    }
28
29    printf("Mission Complete!\n");
30 }
```

Chapter 12

Safety

The companion safety computer must handle communication and hardware failures deterministically:

- **Loss of Companion Link:** If the companion computer stops transmitting heartbeats, the PX4 autopilot immediately triggers a Return-to-Land (RTL) or auto-land failsafe sequence.
- **RC Pilot Manual Takeover:** Pushing any manual override channel stick on the RC transmitter immediately disables Offboard mode, returning aircraft control to the pilot.
- **Loss of GPS Signal:** If GPS signals are lost or the EKF estimate fails during Offboard operations, the vehicle drops to a standard altitude-control mode and initiates a vertical land sequence.

Chapter 13

Configuration

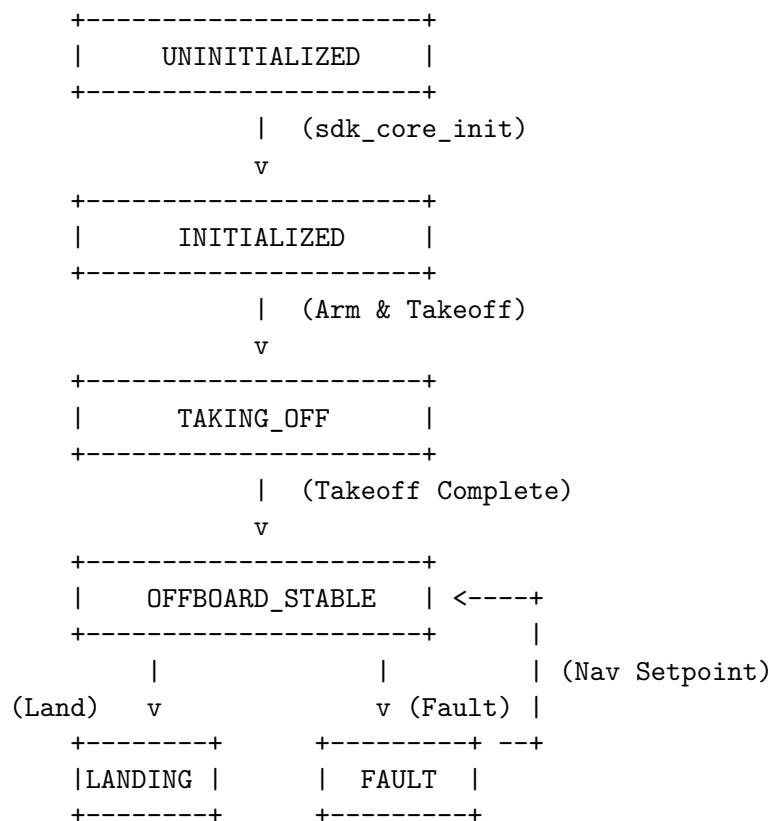
The SDK provides several customizable configuration options to adjust dynamic behavior:

Parameter Name	Type	Default Value	Min / Max Limits	Purpose
CONN_BAUDRATE	Integer	921600	9600 / 3000000	Communication speed over serial.
CONN_DEVICE	String	"/dev/ttyTHS1"	N/A	Target serial interface hardware node.
OFFBOARD_HZ	Float	20.0f	2.0f / 100.0f	Setpoint streaming frequency.
WATCHDOG_MS	Integer	500	100 / 5000	Link drop timeout detection window.

Chapter 14

State Machine

The SDK maintains an internal state machine to track companion computer guidance states:



Chapter 15

Error Codes

This chapter details every SDK error code, mapping return values to meaning, typical cause, recovery recommendations, and severity.

15.1 SDK_OK

- **Meaning:** Target operation executed and completed successfully.
- **Typical Cause:** Expected normal system execution.
- **Recovery Recommendation:** No recovery needed.
- **Severity:** Recoverable.

15.2 SDK_ERR_TIMEOUT

- **Meaning:** An execution deadline was missed or an operation took too long.
- **Typical Cause:** Network latency spikes or unresponsive serial interfaces.
- **Recovery Recommendation:** Verify the physical serial connections; retry the command if safety permits.
- **Severity:** Recoverable.

15.3 SDK_ERR_NOT_READY

- **Meaning:** The flight controller is in a transitional state and cannot process requests.
- **Typical Cause:** EKF2 calculations have not finished calibrating, or there is no GPS lock.
- **Recovery Recommendation:** Wait and verify EKF status registers before re-sending command.
- **Severity:** Recoverable.

15.4 SDK_ERR_INVALID_STATE

- **Meaning:** The command was rejected because it violates active state conditions.
- **Typical Cause:** Attempting a takeoff command when the vehicle is already flying.
- **Recovery Recommendation:** Check state registers via `sdk_core_get_state()` before executing the command.
- **Severity:** Recoverable.

15.5 SDK_ERR_SERIAL

- **Meaning:** Serial connection read or write error.
- **Typical Cause:** Defective wiring, incorrect baud rates, or disconnected serial ports.
- **Recovery Recommendation:** Abort autonomous flight; trigger landing commands or manual takeover.
- **Severity:** Fatal.

15.6 SDK_ERR_ACK_TIMEOUT

- **Meaning:** PX4 failed to respond to a configuration command within the timeout window.
- **Typical Cause:** Heavy traffic on the serial channel or incorrect component ID targets.
- **Recovery Recommendation:** Verify communication link quality and retry configuration command.
- **Severity:** Recoverable.

15.7 SDK_ERR_OFFBOARD_REJECTED

- **Meaning:** PX4 rejected the request to switch into Offboard mode.
- **Typical Cause:** No active, continuous setpoint stream was received before requesting the mode change.
- **Recovery Recommendation:** Verify that offboard stream threads are running at 20Hz before requesting mode switch.
- **Severity:** Recoverable.

15.8 SDK_ERR_ARM_REJECTED

- **Meaning:** Motor arming command was rejected by PX4.
- **Typical Cause:** PX4 pre-flight checks failed (e.g., raw sensor errors, battery levels too low, missing GPS).
- **Recovery Recommendation:** Check diagnostic logs and verify that all sensor calibrations have converged.
- **Severity:** Recoverable.

15.9 SDK_ERR_NO_GPS

- **Meaning:** Inadequate or missing GPS signal.
- **Typical Cause:** Flying indoors or signal blocked by buildings.
- **Recovery Recommendation:** Move the aircraft to a clear outdoor location, or land.
- **Severity:** Fatal (if flying offboard in coordinates mode).

15.10 SDK_ERR_EKF

- **Meaning:** Flight controller Kalman Filter (EKF2) estimates are unhealthy or have failed.
- **Typical Cause:** Magnetic field interference or high accelerometer variance.
- **Recovery Recommendation:** Perform immediate manual pilot override, or land.
- **Severity:** Fatal.

15.11 SDK_ERR_RC_OVERRIDE

- **Meaning:** Mission computer control was overridden by the manual RC pilot.
- **Typical Cause:** Handheld stick movement past 10% deadband threshold.
- **Recovery Recommendation:** Transition companion system to a passive tracking state.
- **Severity:** Recoverable.

15.12 SDK_ERR_PARAMETER

- **Meaning:** The parameter passed to the function exceeds configured limits.
- **Typical Cause:** Command coordinates outside geofence thresholds.
- **Recovery Recommendation:** Check coordinate parameters against geofence limits before calling navigation functions.
- **Severity:** Recoverable.

15.13 SDK_ERR_INTERNAL

- **Meaning:** An internal SDK thread or synchronization error occurred.
- **Typical Cause:** Thread deadlock or resource exhaustion.
- **Recovery Recommendation:** Close process cleanly and restart the companion computer.
- **Severity:** Fatal.

Chapter 16

Thread Safety

To prevent race conditions during asynchronous operations, developers must follow these thread safety rules:

- **Lock Hierarchy:** Telemetry Cache locks must be acquired before Target setpoint locks to avoid deadlocks.
- **Read Rules:** Telemetry fields must be accessed via thread-safe getter APIs. Direct struct access is forbidden.
- **Write Rules:** Writes to outbound command parameters must occur inside the Target setpoint lock boundaries.

Chapter 17

PX4 Integration

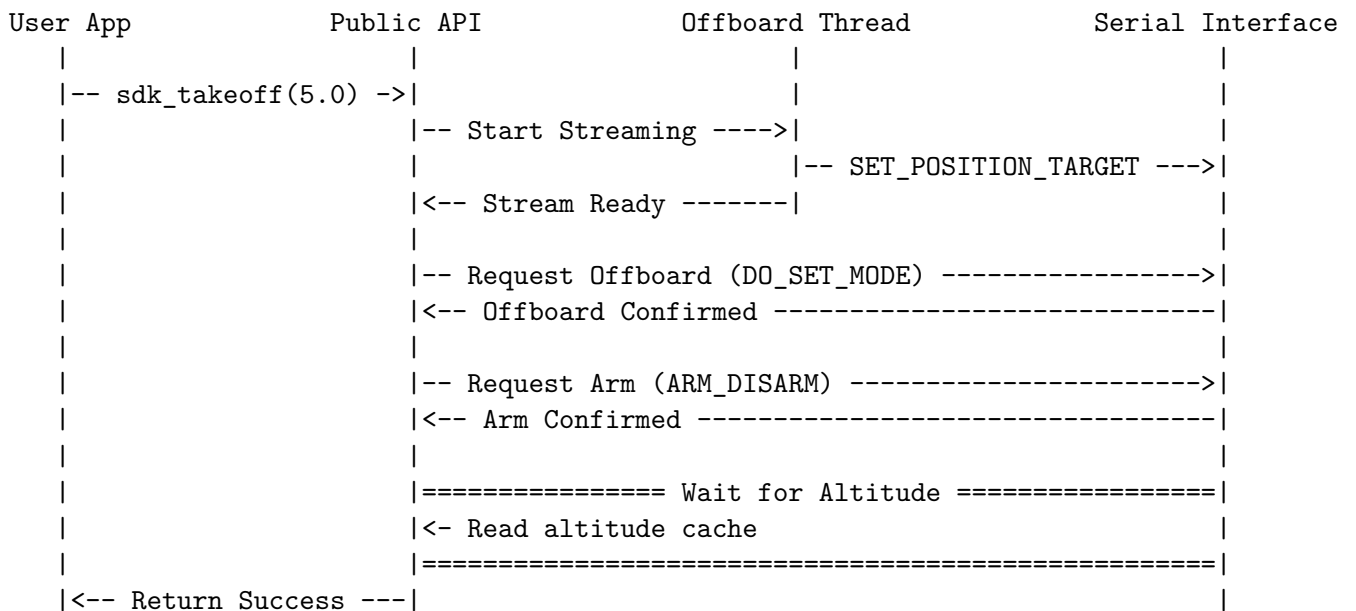
The SDK is certified and tested against the following PX4 configurations:

- **Autopilot Core:** PX4 Autopilot Middleware v1.14 Stable Release.
- **State Estimator:** EKF2 (Extended Kalman Filter 2).
- **Control Protocol:** MAVLink Custom Autopilot Dialect (PX4 Profile).

Chapter 18

Internal Operation

The sequence chart below outlines the execution steps and thread coordination during a takeoff call:



Chapter 19

Best Practices

- **Maintain constant communication rate:** Ensure setpoint updates are sent at a steady 20Hz rate to prevent flight controller timeouts.
- **Process frame transformations with care:** Always verify your local coordinate systems and frames match before translating coordinate targets.
- **Implement validation checks:** Always verify system health telemetry values (GPS status, EKF health, battery voltage) prior to armed maneuvers.

Chapter 20

FAQ

Q: Why did my offboard command switch fail?

A: Check that you are streaming active setpoint packets (such as SET_POSITION_TARGET_LOCAL_NED) at 20Hz before requesting offboard mode.

Q: How does RC manual takeover override offboard control?

A: Stick movements past the 10% threshold tell the PX4 autopilot to drop offboard control and revert to manual mode. The SDK watchdog thread detects this mode change and transitions gracefully.

Chapter 21

Glossary

- **PX4:** An open-source flight control software platform for drones and other unmanned vehicles.
- **MAVLink:** A lightweight messaging protocol used to communicate with drones and other unmanned systems.
- **Offboard Mode:** A flight mode where the drone accepts external position, velocity, or attitude commands.
- **EKF2:** The second-generation Extended Kalman Filter algorithm used to estimate vehicle altitude and position.

Chapter 22

Appendix

22.1 System Return and Error State Codes

The following table summarizes the error mappings used statically by the SDK:

Error Symbol	Numerical Value	Context
SDK_ERR_SUCCESS	0	Target operation executed successfully.
SDK_ERR_LINK_DOWN	-101	Serial connection lost or timeout detected.
SDK_ERR_NOT_ARMED	-102	Arm check failed prior to takeoff maneuver.
SDK_ERR_BAD_COORDINATE	-103	Trajectory coordinates fell outside range boundaries.
SDK_ERR_OFFBOARD_TIMEOUT	-104	Auto navigation fell back to manual hold mode.

Table 22.1: System Return and Error State Codes

22.2 Which Function Should I Use?

To streamline spatial task transitions, developers should reference the decision map below to match mission objectives with target APIs:

If you want to...	Target API Call	Subsystem Module
Fly to absolute GPS Coordinates	gps_go()	Navigation
Follow a dynamic moving object	set_velocity()	Velocity
Hover indefinitely in place	hold_alt() / hover()	Altitude / Flight
Hover for a fixed duration	hover(duration_ms)	Flight
Rotate to an absolute compass heading	yaw_to()	Yaw
Rotate at a constant angular rate	yaw_rate()	Yaw
Increase altitude by delta offset	change_alt()	Altitude
Set an absolute flight altitude target	set_alt()	Altitude
Pause autopilot mission waypoint sequence	pause_mission()	Mission
Recover control manually immediately	RC takeover / clear_rc_takeover()	RC
Stop flight travel immediately	emergency_brake()	Flight

Table 22.2: Operational Objective Lookup Map